

Phase 3: Insertion and Insertion Sort — Full Structured Beginner Note

1. Current Progress Before Phase 3

Before Phase 3, you already studied two important phases.

Phase 1: Sorting Foundations

In Phase 1, you learned the basic language used to understand sorting algorithms.

You learned words like:

Term	Simple meaning
Sorting	Arranging data in order
Ascending order	Smallest to largest
Descending order	Largest to smallest
Comparison	Checking two values to decide their order
Swap	Exchanging two values
Shift	Moving values to make space
Adaptive	Faster when data is already or nearly sorted
Extra memory	Additional space used by an algorithm
Stable	Equal values keep their original relative order
Big O	Describes how algorithm work grows
$O(1)$	Constant time or constant space
$O(n)$	Linear time
$O(n^2)$	Quadratic time
Pointer	A variable that stores an address

The most important lesson from Phase 1 was:

Do not judge a sorting algorithm only by asking, “Does it sort?”

A better question is:

How does it sort, how much time does it take, how much memory does it use, and does it preserve duplicate order?

Phase 2: Bubble Sort

In Phase 2, you learned Bubble Sort.

Bubble Sort works by comparing neighboring elements.

Example:

8 5

Since $8 > 5$, Bubble Sort swaps them:

5 8

Bubble Sort repeats this process many times.

Important points from Bubble Sort:

- It compares adjacent elements.
- It swaps values when they are in the wrong order.
- One full scan is called a pass.
- After each pass, one large value is fixed at the end.
- After k passes, the k largest values are fixed at the end.
- It can be adaptive if we use a `flag`.
- It is stable if we use $>$ instead of $>=$.
- Best-case time is $O(n)$ when the adaptive flag is used.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.

2. Why Phase 3 Comes After Bubble Sort

Bubble Sort moves values mainly by repeated neighboring swaps.

Insertion Sort uses a different idea.

Instead of repeatedly swapping neighbors, Insertion Sort:

Takes one value and inserts it into the correct position inside a sorted part of the array.

This is why Phase 3 is called:

Insertion and Insertion Sort

In this phase, you will learn:

- Insertion into a sorted array
- Insertion into a sorted linked list
- Insertion Sort logic
- Insertion Sort dry run
- Insertion Sort C program
- The `j = -1` edge case
- Best-case and worst-case time
- Space complexity
- Adaptiveness
- Stability
- Bubble Sort vs Insertion Sort
- Common beginner mistakes

3. Main Goal of Phase 3

The main goal of Phase 3 is to understand this idea:

If one part is already sorted, we can take the next value and insert it into the correct place inside that sorted part.

This idea is used in Insertion Sort.

Insertion Sort is important because it is:

- Easy to understand
- Good for small arrays
- Good for nearly sorted arrays
- Naturally adaptive
- Stable
- In-place
- Suitable for linked lists

4. Real-Life Analogy: Arranging Playing Cards

Imagine you are holding playing cards in your hand.

You pick up cards one by one.

Your left hand is already sorted.

When you pick up a new card, you compare it with the cards in your hand and place it in the correct position.

Example:

Cards in hand: 3 7 10
New card: 5

You place 5 between 3 and 7:

3 5 7 10

That is the idea of Insertion Sort.

5. What Does “Insertion” Mean?

Insertion means placing a new value into its correct position.

Example:

2 6 10 15 20

This list is already sorted.

Now suppose we want to insert:

12

The correct position is between 10 and 15.

Final result:

2 6 10 12 15 20

So insertion means:

Put a value where it belongs.

6. Insertion into a Sorted Array

Before learning full Insertion Sort, we first understand how to insert one value into a sorted array.

Suppose we have this sorted array:

2 6 10 15 20 25

We want to insert:

12

The correct position is:

2 6 10 12 15 20 25

But there is a problem.

Arrays store elements in continuous memory. We cannot simply place 12 between 10 and 15 unless we make space.

So we shift larger elements to the right.

7. Dry Run: Insert 12 into a Sorted Array

Original sorted array:

2 6 10 15 20 25

Value to insert:

12

Start from the right side.

Step 1: Compare 25 and 12

25 > 12, so shift 25 right.

2 6 10 15 20 _ 25

Step 2: Compare 20 and 12

20 > 12, so shift 20 right.

2 6 10 15 _ 20 25

Step 3: Compare 15 and 12

15 > 12, so shift 15 right.

2 6 10 _ 15 20 25

Step 4: Compare 10 and 12

10 > 12 is false.

So we stop shifting.

Now insert 12 after 10.

2 6 10 12 15 20 25

8. Important Idea from Array Insertion

When inserting into a sorted array:

1. Start from the right side.
2. Shift all bigger values one step right.
3. Stop when you find a smaller or equal value.
4. Insert the new value in the empty position.

This is the same logic used inside Insertion Sort.

9. Simple Code: Insert into a Sorted Array

```
void insertInSortedArray(int A[], int *n, int capacity, int x) {
    if (*n >= capacity) {
        printf("Array is full. Cannot insert.\n");
        return;
    }

    int i = *n - 1;

    while (i >= 0 && A[i] > x) {
        A[i + 1] = A[i];
        i--;
    }

    A[i + 1] = x;
    (*n)++;
}
```

10. Explanation of Sorted Array Insertion Code

Function header

```
void insertInSortedArray(int A[], int *n, int capacity, int x)
```

Part	Meaning
A[]	The array
int *n	Pointer to the current number of elements
capacity	Maximum size of the array
x	The value to insert

Capacity check

```
if (*n >= capacity) {
    printf("Array is full. Cannot insert.\n");
    return;
}
```

This checks whether the array has enough space.

If the array is full, we cannot insert a new value.

Start from the last element

```
int i = *n - 1;
```

If the array has `n` elements, the last used index is `n - 1`.

Shift bigger values

```
while (i >= 0 && A[i] > x) {  
    A[i + 1] = A[i];  
    i--;  
}
```

This shifts all values bigger than `x` one position to the right.

Insert the new value

```
A[i + 1] = x;
```

When the loop stops, `i + 1` is the correct position for `x`.

Increase the size

```
(*n)++;
```

After inserting one new value, the number of elements increases by 1.

11. Insertion into a Sorted Linked List

Insertion can also happen in a linked list.

A linked list does not store values continuously like an array.

Instead, each node stores:

- Data
- A link to the next node

Example:

2 -> 6 -> 10 -> 15 -> 20

Now suppose we want to insert:

18

The correct position is between 15 and 20 .

Final list:

2 -> 6 -> 10 -> 15 -> 18 -> 20

12. Difference Between Array Insertion and Linked List Insertion

Array insertion	Linked list insertion
Requires shifting values	Does not require shifting values
Values are stored in continuous memory	Nodes are connected using links
May be costly if many values shift	Only links are changed
Need free space in the array	Need memory for a new node

13. Pointers Used in Linked List Insertion

Linked list insertion commonly uses three pointers:

Pointer	Meaning
p	Moves through the list
q	Follows behind p
t	New node to insert

The idea is:

- `p` searches for the first node greater than the value.
- `q` stays one node behind `p`.
- `t` is inserted between `q` and `p`.

14. Dry Run: Insert 18 into a Sorted Linked List

Original list:

```
2 -> 6 -> 10 -> 15 -> 20
```

Value to insert:

```
18
```

Start:

```
p = first node  
q = NULL
```

Step 1

`p` points to `2`.

Since `2 < 18`, move forward.

```
q = 2  
p = 6
```

Step 2

`p` points to `6`.

Since `6 < 18`, move forward.

```
q = 6  
p = 10
```

Step 3

p points to 10.

Since $10 < 18$, move forward.

```
q = 10  
p = 15
```

Step 4

p points to 15.

Since $15 < 18$, move forward.

```
q = 15  
p = 20
```

Step 5

p points to 20.

$20 < 18$ is false.

So stop.

Insert 18 between q and p.

```
15 -> 18 -> 20
```

Final list:

```
2 -> 6 -> 10 -> 15 -> 18 -> 20
```

15. Code: Insert into a Sorted Linked List

```
#include <stdio.h>  
#include <stdlib.h>
```

```

struct Node {
    int data;
    struct Node *next;
};

void insertInSortedLinkedList(struct Node **first, int x) {
    struct Node *t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;
    t->next = NULL;

    struct Node *p = *first;
    struct Node *q = NULL;

    while (p != NULL && p->data < x) {
        q = p;
        p = p->next;
    }

    if (q == NULL) {
        t->next = *first;
        *first = t;
    } else {
        t->next = p;
        q->next = t;
    }
}

```

16. Explanation of Linked List Insertion Code

Node structure

```

struct Node {
    int data;
    struct Node *next;
};

```

Each node contains:

- **data** : the value
- **next** : address of the next node

Create a new node

```
struct Node *t = (struct Node *)malloc(sizeof(struct Node));  
t->data = x;  
t->next = NULL;
```

This creates a new node and stores `x` inside it.

Set traversal pointers

```
struct Node *p = *first;  
struct Node *q = NULL;
```

`p` starts at the first node.

`q` starts as `NULL` because there is no previous node yet.

Move through the list

```
while (p != NULL && p->data < x) {  
    q = p;  
    p = p->next;  
}
```

This loop moves forward while the current value is smaller than `x`.

Case 1: Insert at the beginning

```
if (q == NULL) {  
    t->next = *first;  
    *first = t;  
}
```

If `q` is still `NULL`, the new value belongs at the beginning.

Case 2: Insert in the middle or end

```
else {  
    t->next = p;  
    q->next = t;  
}
```

This inserts `t` between `q` and `p`.

17. What Is Insertion Sort?

Insertion Sort is a sorting algorithm that builds a sorted part one value at a time.

It starts by treating the first element as already sorted.

Example:

8 5 7 3 2

At the beginning:

[8] 5 7 3 2

The sorted part is:

[8]

Then Insertion Sort takes the next value, `5`, and inserts it into the correct position inside the sorted part.

After inserting `5`:

[5 8] 7 3 2

Then it inserts `7`:

[5 7 8] 3 2

Then it inserts `3`:

[3 5 7 8] 2

Then it inserts 2:

[2 3 5 7 8]

Final sorted array:

2 3 5 7 8

18. Main Idea of Insertion Sort

Insertion Sort works like this:

1. Treat the first element as sorted.
2. Pick the next element.
3. Compare it with elements on the left.
4. Shift bigger elements to the right.
5. Insert the picked element into the empty space.
6. Repeat until the whole array is sorted.

Short memory trick:

Pick, shift, insert.

19. Insertion Sort Pseudocode

```
for i from 1 to n - 1:
    x = A[i]
    j = i - 1

    while j >= 0 and A[j] > x:
        A[j + 1] = A[j]
        j = j - 1

    A[j + 1] = x
```

20. Why Insertion Sort Starts at Index 1

Insertion Sort starts at index `1`, not index `0`.

Reason:

The first element alone is already sorted.

Example:

```
8 5 7 3 2
```

At the start:

```
[8] 5 7 3 2
```

A single value is always sorted by itself.

So the first value we need to insert is at index `1`.

21. Important Variables in Insertion Sort

Variable	Meaning
<code>A[]</code>	The array
<code>n</code>	Number of elements
<code>i</code>	Index of the value currently being inserted
<code>x</code>	The value being inserted
<code>j</code>	Moves left through the sorted part
<code>A[j]</code>	Current value in the sorted part
<code>A[j + 1]</code>	Position where shifting or insertion happens

22. Why We Store `A[i]` in `x`

This line is very important:


```
x = A[i];
```

We store `A[i]` in `x` before shifting.

Why?

Because shifting values to the right may overwrite `A[i]`.

Example:

```
5 8 7
```

We want to insert `7`.

If we shift `8` into the position of `7`, the original `7` disappears unless we saved it first.

So we save it:

```
x = A[i];
```

Now even if `A[i]` is overwritten, the value is safe inside `x`.

23. Insertion Sort Core Code

```
void insertionSort(int A[], int n) {
    int i, j, x;

    for (i = 1; i < n; i++) {
        x = A[i];
        j = i - 1;

        while (j >= 0 && A[j] > x) {
            A[j + 1] = A[j];
            j--;
        }

        A[j + 1] = x;
    }
}
```

24. Explanation of Insertion Sort Code

Function header

```
void insertionSort(int A[], int n)
```

This function receives:

Parameter	Meaning
A[]	Array to sort
n	Number of elements

Variables

```
int i, j, x;
```

Variable	Meaning
i	Selects the next value to insert
j	Moves left through the sorted part
x	Temporarily stores the selected value

Outer loop

```
for (i = 1; i < n; i++)
```

The loop starts at 1 because the first element is already considered sorted.

Store the value

```
x = A[i];
```

This saves the value to insert.

Start checking from the left side

```
j = i - 1;
```

This means `j` starts from the element just before `i`.

Shift bigger values

```
while (j >= 0 && A[j] > x) {  
    A[j + 1] = A[j];  
    j--;  
}
```

This loop shifts bigger values to the right.

It continues while:

1. `j >= 0`, meaning we are still inside the array.
 2. `A[j] > x`, meaning the left value is bigger than the value being inserted.
-

Insert `x`

```
A[j + 1] = x;
```

When the loop stops, `j + 1` is the correct position for `x`.

25. Full Dry Run of Insertion Sort

Let us sort:

```
8 5 7 3 2
```

Final answer should be:

```
2 3 5 7 8
```

26. Initial State

Original array:

8 5 7 3 2

Treat the first element as sorted:

[8] 5 7 3 2

Sorted part:

8

Unsorted part:

5 7 3 2

27. Pass 1: Insert 5

Current array:

[8] 5 7 3 2

Value to insert:

x = 5

j starts at index 0.

Compare:

8 > 5

True, so shift 8 right.

_ 8 7 3 2

Now `j` becomes `-1`, so stop.

Insert `5` at index `0`.

5 8 7 3 2

Sorted part:

[5 8]

28. Pass 2: Insert 7

Current array:

[5 8] 7 3 2

Value to insert:

`x = 7`

`j` starts at index `1`.

Compare:

`8 > 7`

True, so shift `8` right.

5 _ 8 3 2

Now `j` moves to index `0`.

Compare:

5 > 7

False.

Stop shifting.

Insert 7 after 5.

5 7 8 3 2

Sorted part:

[5 7 8]

29. Pass 3: Insert 3

Current array:

[5 7 8] 3 2

Value to insert:

x = 3

Compare:

8 > 3

True, shift 8 right.

5 7 _ 8 2

Compare:

7 > 3

True, shift 7 right.

5 _ 7 8 2

Compare:

5 > 3

True, shift 5 right.

_ 5 7 8 2

Now $j = -1$, so stop.

Insert 3 at index 0.

3 5 7 8 2

Sorted part:

[3 5 7 8]

30. Pass 4: Insert 2

Current array:

[3 5 7 8] 2

Value to insert:

x = 2

Compare:

8 > 2

True, shift 8 right.

3 5 7 _ 8

Compare:

7 > 2

True, shift 7 right.

3 5 _ 7 8

Compare:

5 > 2

True, shift 5 right.

3 _ 5 7 8

Compare:

3 > 2

True, shift 3 right.

_ 3 5 7 8

Now $j = -1$, so stop.

Insert 2 at index 0.

2 3 5 7 8

Final sorted array:

2 3 5 7 8

31. Summary of the Dry Run

Original array:

8 5 7 3 2

After pass 1:

5 8 7 3 2

After pass 2:

5 7 8 3 2

After pass 3:

3 5 7 8 2

After pass 4:

2 3 5 7 8

Final sorted array:

2 3 5 7 8

32. Important Difference from Bubble Sort Passes

In Bubble Sort:

After each pass, one largest value is fixed at the end.

In Insertion Sort:

After each pass, the left part becomes sorted, but the values may not yet be in their final global positions.

Example:

Original:

8 5 7 3 2

After inserting 5:

5 8 7 3 2

The left part [5 8] is sorted.

But 5 and 8 are not necessarily in their final positions because smaller values like 3 and 2 still exist on the right.

So do not confuse Bubble Sort passes with Insertion Sort passes.

33. Complete Insertion Sort Program in C

```
#include <stdio.h>

void insertionSort(int A[], int n) {
    int i, j, x;

    for (i = 1; i < n; i++) {
        x = A[i];          // value we want to insert
        j = i - 1;         // start checking from the left side

        while (j >= 0 && A[j] > x) {
            A[j + 1] = A[j]; // shift bigger value right
            j--;
        }

        A[j + 1] = x;      // insert x in the correct position
    }
}

int main() {
    int A[] = {8, 5, 7, 3, 2};
```

```

int n = sizeof(A) / sizeof(A[0]);

insertionSort(A, n);

printf("Sorted array: ");
for (int i = 0; i < n; i++) {
    printf("%d ", A[i]);
}

printf("\n");
return 0;
}

```

Expected output:

```
Sorted array: 2 3 5 7 8
```

34. Full Phase 3 Program: Array Insertion, Linked List Insertion, and Insertion Sort

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void insertionSort(int A[], int n) {
    int i, j, x;

    for (i = 1; i < n; i++) {
        x = A[i];
        j = i - 1;

        while (j >= 0 && A[j] > x) {
            A[j + 1] = A[j];
            j--;
        }

        A[j + 1] = x;
    }
}

```

```

void insertInSortedArray(int A[], int *n, int capacity, int x) {
    if (*n >= capacity) {
        printf("Array is full. Cannot insert.\n");
        return;
    }

    int i = *n - 1;

    while (i >= 0 && A[i] > x) {
        A[i + 1] = A[i];
        i--;
    }

    A[i + 1] = x;
    (*n)++;
}

void insertInSortedLinkedList(struct Node **first, int x) {
    struct Node *t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;
    t->next = NULL;

    struct Node *p = *first;
    struct Node *q = NULL;

    while (p != NULL && p->data < x) {
        q = p;
        p = p->next;
    }

    if (q == NULL) {
        t->next = *first;
        *first = t;
    } else {
        t->next = p;
        q->next = t;
    }
}

void displayArray(int A[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");
}

void displayList(struct Node *p) {

```

```

    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
    printf("\n");
}

int main() {
    int A[20] = {11, 13, 7, 12, 16, 9, 24, 5, 10, 3};
    int n = 10;

    insertionSort(A, n);
    printf("Insertion sorted array: ");
    displayArray(A, n);

    int B[10] = {2, 6, 10, 15, 20, 25};
    int m = 6;

    insertInSortedArray(B, &m, 10, 12);
    printf("After inserting 12: ");
    displayArray(B, m);

    struct Node *first = NULL;

    insertInSortedLinkedList(&first, 2);
    insertInSortedLinkedList(&first, 6);
    insertInSortedLinkedList(&first, 10);
    insertInSortedLinkedList(&first, 15);
    insertInSortedLinkedList(&first, 20);
    insertInSortedLinkedList(&first, 18);

    printf("Linked list after inserting 18: ");
    displayList(first);

    return 0;
}

```

35. Expected Output of Full Program

```

Insertion sorted array: 3 5 7 9 10 11 12 13 16 24
After inserting 12: 2 6 10 12 15 20 25
Linked list after inserting 18: 2 6 10 15 18 20

```

36. The Dangerous Edge Case: $j = -1$

This is one of the most important beginner points in Insertion Sort.

Suppose we want to insert 2 into this sorted part:

6 8

We want the final result:

2 6 8

Start:

$x = 2$
 $j = 1$

Compare:

$A[1] > 2$
 $8 > 2$

True, so shift 8 right.

Now:

$j = 0$

Compare:

$A[0] > 2$
 $6 > 2$

True, so shift 6 right.

Now:

```
j = -1
```

There is no index `-1` in the array.

So we must stop before trying to read:

```
A[-1]
```

That is why this condition is correct:

```
while (j >= 0 && A[j] > x)
```

37. Why the Order of the While Condition Matters

Correct:

```
while (j >= 0 && A[j] > x)
```

Wrong:

```
while (A[j] > x && j >= 0)
```

Why is the second one dangerous?

Because C checks the left side first.

If `j` is already `-1`, the program may try to check:

```
A[-1] > x
```

That is invalid memory access.

So always write:

```
j >= 0
```

before:

$A[j] > x$

38. Best-Case Time Complexity

The best case happens when the array is already sorted.

Example:

2 3 5 7 8

For each value, Insertion Sort checks the left value.

The condition:

$A[j] > x$

is false immediately.

So no shifting is needed.

The algorithm only makes a small number of comparisons.

Best-case time:

$O(n)$

39. Why the Best Case Is $O(n)$

If the array has n elements, the outer loop runs about $n - 1$ times.

But the inner `while` loop stops immediately each time.

So the work grows roughly in direct proportion to n .

That is why the best case is:

$O(n)$

40. Worst-Case Time Complexity

The worst case happens when the array is reverse sorted.

Example:

8 7 5 3 2

In this case, every new value must move all the way to the front.

For example:

- 7 moves before 8.
- 5 moves before 7 and 8.
- 3 moves before 5, 7, and 8.
- 2 moves before all previous values.

So many shifts are needed.

Worst-case time:

$O(n^2)$

41. Why the Worst Case Is $O(n^2)$

In reverse order, the amount of shifting grows like this:

$1 + 2 + 3 + \dots + (n - 1)$

This sum grows like:

n^2

So worst-case time is:

$O(n^2)$

42. Space Complexity

Insertion Sort does not need another large array.

It sorts inside the original array.

It only uses a few variables:

i, j, x

So space complexity is:

$O(1)$

This means constant extra space.

43. Is Insertion Sort In-Place?

Yes.

Insertion Sort is in-place because it sorts mainly inside the original array.

It does not create another array to store the sorted result.

It only uses a few extra variables.

So Insertion Sort is:

In-place

44. Is Insertion Sort Adaptive?

Yes.

Insertion Sort is naturally adaptive.

Adaptive means:

The algorithm becomes faster when the data is already sorted or nearly sorted.

Insertion Sort does this naturally because the `while` loop stops early when values are already in order.

Example:

2 3 5 7 8

When inserting each value, the left side is already smaller.

So no shifting is needed.

That is why Insertion Sort is very good for nearly sorted data.

45. Bubble Sort Adaptiveness vs Insertion Sort Adaptiveness

Bubble Sort needs a `flag` to become adaptive.

Insertion Sort does not need a flag.

Algorithm	How it becomes adaptive
Bubble Sort	Needs a <code>flag</code> to stop early
Insertion Sort	Naturally adaptive because shifting stops early

46. Is Insertion Sort Stable?

Yes.

Insertion Sort is stable if we use this condition:

$A[j] > x$

It is stable because it does not move equal values past each other.

47. Stability Example

Suppose we have two equal values:

5A 5B

5A came before 5B originally.

When inserting 5B, the algorithm checks:

5A > 5B

This is false because both values are equal.

So 5A is not shifted after 5B.

Final order stays:

5A 5B

So Insertion Sort is stable.

48. Why Using `>=` Breaks Stability

Correct condition:

```
while (j >= 0 && A[j] > x)
```

Wrong condition:

```
while (j >= 0 && A[j] >= x)
```

If we use `>=`, equal values may be shifted.

Example:

5A 5B

Using `>=`:

5A `>=` 5B

This becomes true because they are equal.

So 5A may shift after 5B.

Result:

5B 5A

Now the original order is broken.

So using `>=` makes the algorithm unstable.

49. Insertion Sort Properties

Property	Insertion Sort result
Sorting type	Comparison-based sorting
Movement type	Shifting
Best-case time	$O(n)$
Worst-case time	$O(n^2)$
Space complexity	$O(1)$
In-place?	Yes
Adaptive?	Yes
Stable?	Yes, if using <code>></code>
Good for nearly sorted data?	Yes
Good for linked lists?	Yes
Good for huge random data?	Usually no

50. Bubble Sort vs Insertion Sort

Feature	Bubble Sort	Insertion Sort
Main idea	Push large values to the end	Insert each value into sorted left part
Main movement	Swapping neighboring elements	Shifting bigger elements right
Starts with	Whole array unsorted	First element treated as sorted
Pass result	One largest value fixed at the end	Left part becomes sorted
Adaptive?	Yes, with <code>flag</code>	Yes, naturally
Stable?	Yes, if using <code>></code>	Yes, if using <code>></code>
Best case	$O(n)$ with adaptive flag	$O(n)$
Worst case	$O(n^2)$	$O(n^2)$
Extra memory	$O(1)$	$O(1)$
Good for nearly sorted data?	Yes, if adaptive	Very good
Linked-list friendly?	Not especially	Yes

51. Which Is Better: Bubble Sort or Insertion Sort?

For learning, Bubble Sort is easier to visualize because it uses adjacent swaps.

But in practice, Insertion Sort is often better for small or nearly sorted data.

Why?

Because Insertion Sort:

- Does not need a flag to be adaptive
- Usually does fewer unnecessary operations on nearly sorted data
- Uses shifting instead of repeated swapping
- Works well with linked lists

So for nearly sorted data, Insertion Sort is usually preferred.

52. Insertion Sort and Linked Lists

Insertion Sort is suitable for linked lists.

Why?

Because insertion in a linked list does not require shifting many values.

In an array, to insert a value, we may need to shift many elements right.

In a linked list, we only change links.

Example:

```
2 -> 6 -> 10 -> 15 -> 20
```

Insert 18:

```
2 -> 6 -> 10 -> 15 -> 18 -> 20
```

Only links around 15, 18, and 20 need to change.

53. Common Beginner Mistakes

Mistake 1: Forgetting to store `A[i]` in `x`

Wrong idea:

```
while (j >= 0 && A[j] > A[i]) {  
    A[j + 1] = A[j];  
    j--;  
}
```

This is dangerous because `A[i]` may be overwritten during shifting.

Correct idea:

```
x = A[i];
```

Save the value first.

Mistake 2: Writing the while condition in the wrong order

Wrong:

```
while (A[j] > x && j >= 0)
```

Correct:

```
while (j >= 0 && A[j] > x)
```

Always check `j >= 0` first.

Mistake 3: Using `>=` instead of `>`

Wrong:

```
while (j >= 0 && A[j] >= x)
```

Correct:

```
while (j >= 0 && A[j] > x)
```

Using `>=` can break stability.

Mistake 4: Thinking each pass fixes the largest value

That is true for Bubble Sort, not Insertion Sort.

In Insertion Sort, each pass makes the left part sorted.

Mistake 5: Forgetting to insert `x` after shifting

Wrong:

```
while (j >= 0 && A[j] > x) {  
    A[j + 1] = A[j];
```



```
j--;  
}
```

This shifts values but never places `x` back.

Correct:

```
A[j + 1] = x;
```

Mistake 6: Starting the outer loop at 0

Less useful:

```
for (i = 0; i < n; i++)
```

Correct:

```
for (i = 1; i < n; i++)
```

The first element is already sorted by itself.

Mistake 7: Not handling insertion at the head of a linked list

In linked-list insertion, if the new value belongs at the beginning, `q` will be `NULL`.

So we must handle this case:

```
if (q == NULL) {  
    t->next = *first;  
    *first = t;  
}
```

54. Key Vocabulary for Phase 3

Term	Simple meaning
Insertion	Placing a value into its correct position

Term	Simple meaning
Sorted part	The part of the array that is already sorted
Unsorted part	The part still waiting to be inserted
Shift	Move a value one position to make space
<code>x</code>	The value being inserted
<code>i</code>	Index of the selected value
<code>j</code>	Moves left through the sorted part
Stable	Equal values keep their original order
Adaptive	Faster when data is already or nearly sorted
In-place	Sorts inside the original array
Linked list insertion	Inserting by changing links instead of shifting values
<code>p</code>	Pointer that moves through a linked list
<code>q</code>	Pointer that follows behind <code>p</code>
<code>t</code>	New node to insert

55. Insertion Sort Evaluation Checklist

When studying Insertion Sort, ask these questions.

Comparisons

Does it compare values?

Yes.

It compares:

$A[j] > x$

Movement

Does it move values?

Yes.

It moves values by shifting, not by repeated swapping.

Adaptiveness

Is it adaptive?

Yes.

It is naturally adaptive because the inner loop stops early when the data is already sorted or nearly sorted.

Memory

Does it need a large extra array?

No.

It uses only a few variables.

Space complexity:

$O(1)$

Stability

Is it stable?

Yes, if we use:

$A[j] > x$

not:

$A[j] \geq x$

Best case

What is the best case?

Already sorted data.

Best-case time:

$O(n)$

Worst case

What is the worst case?

Reverse sorted data.

Worst-case time:

$O(n^2)$

56. Phase 3 Summary

Phase 3 teaches insertion and Insertion Sort.

The main idea is:

Take the next value and insert it into the correct position inside the sorted left part.

Important points:

- Insertion means placing a value where it belongs.
- In arrays, insertion may require shifting values right.
- In linked lists, insertion requires changing links.
- Insertion Sort treats the first element as already sorted.
- It starts from index `1`.
- It stores the selected value in `x`.
- It uses `j` to move left through the sorted part.
- It shifts bigger values to the right.
- It inserts `x` at `j + 1`.
- The condition must be `j >= 0 && A[j] > x`.
- Checking `j >= 0` first prevents invalid access like `A[-1]`.
- Best-case time is $O(n)$.
- Worst-case time is $O(n^2)$.
- Space complexity is $O(1)$.

- It is in-place.
 - It is naturally adaptive.
 - It is stable if we use `>` instead of `>=`.
 - It is very good for nearly sorted data.
 - It is suitable for linked lists.
-

57. One-Sentence Memory Trick

Remember Insertion Sort like this:

Pick the next value, shift bigger values right, and insert it into the sorted left side.

Even shorter:

Pick, shift, insert.

58. What Comes Next

After Phase 3, you are ready for Phase 4: Selection Sort.

Insertion Sort builds a sorted part by inserting each value.

Selection Sort will use a different idea:

Select the smallest value from the unsorted part and place it in the correct position.

So far, you know:

Phase	Algorithm / Topic	Main idea
Phase 1	Foundations	Learn sorting vocabulary and evaluation criteria
Phase 2	Bubble Sort	Push large values to the end using adjacent swaps
Phase 3	Insertion Sort	Insert each value into the sorted left part

Phase 4 will continue the comparison of simple $O(n^2)$ sorting algorithms.